

RAG 技术详解与实战应用

第18讲：高阶RAG：Agentic RAG



目录

-  1. 上节回顾
-  2. Agentic RAG基本概念
-  3. AI Agent基本概念
-  4. Agentic RAG基础结构及实践
-  5. 多模态Agentic RAG论文系统



权限与数据隔离

- **部门级隔离**：支持按部门/项目独立管理知识库，确保数据安全。
- **标签体系**：通过元数据实现细粒度权限控制，动态过滤非授权内容。
- **文档管理服务**：提供Web界面或API，支持文档增删改查，适配企业定制化需求。

灵活共享与多用户并发

- **算法复用**：全局算法池（解析/嵌入/排序模型）支持跨部门调用，适配不同业务场景。
- **知识库解耦**：一个文档管理服务可管理多个知识库，一个RAG服务可检索多知识库，实现多对多协作。
- **多用户隔离**：通过globals配置中心管理会话历史，session_id隔离对话流，支持高并发流式输出。

全链路安全保障

- **企业安全**：数据加密、私有化部署、信创兼容（国产软硬件）。
- **公共安全**：敏感词过滤（如DFA算法）、涉政/暴恐内容拦截，合规输出。
- **敏感内容处理**：上传/检索/生成阶段自动屏蔽敏感信息（如合同条款）。

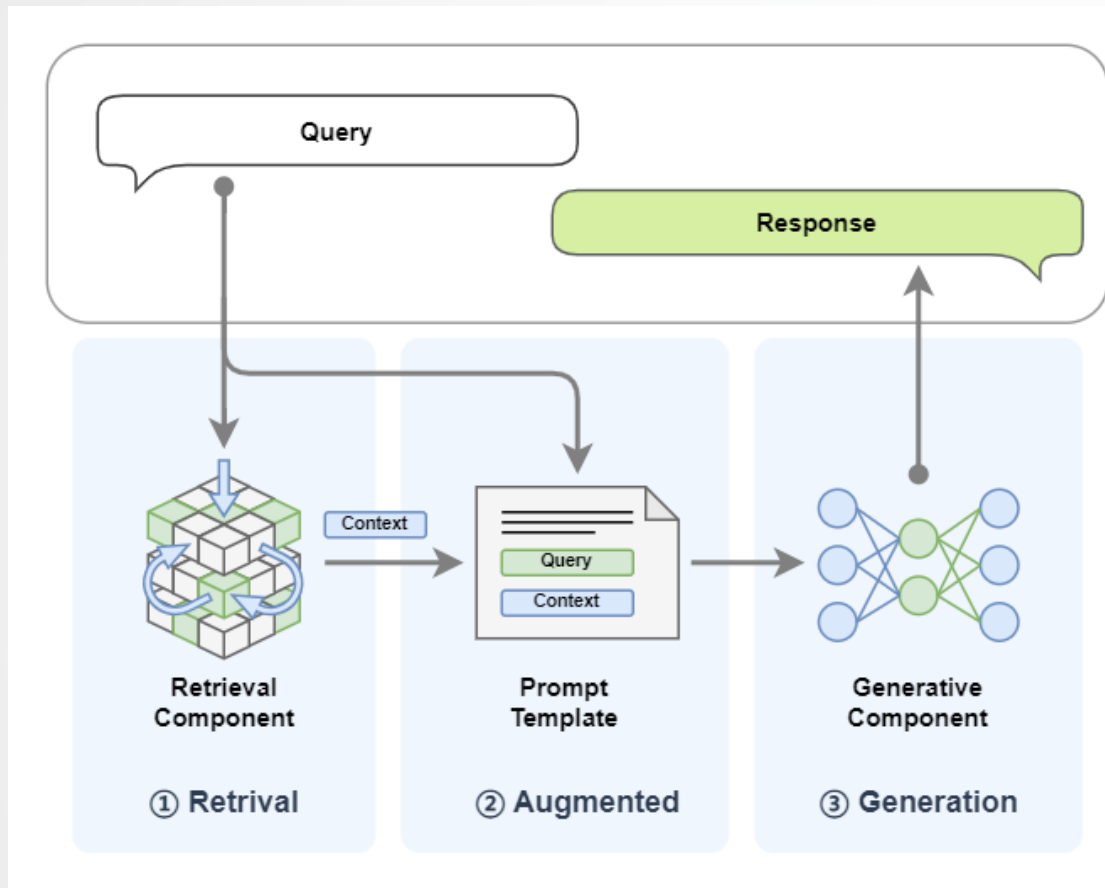


目录

-  1. 上节回顾
-  2. Agentic RAG基本概念
-  3. AI Agent基本概念
-  4. Agentic RAG基础结构及实践
-  5. 多模态Agentic RAG论文系统



RAG基础回顾



检索增强生成（Retrieval-Augmented Generation，简称RAG）技术是一种利用外挂知识源为大语言模型补充上下文来强化输入从而提高大语言模型生成内容质量，并减少幻觉的技术。

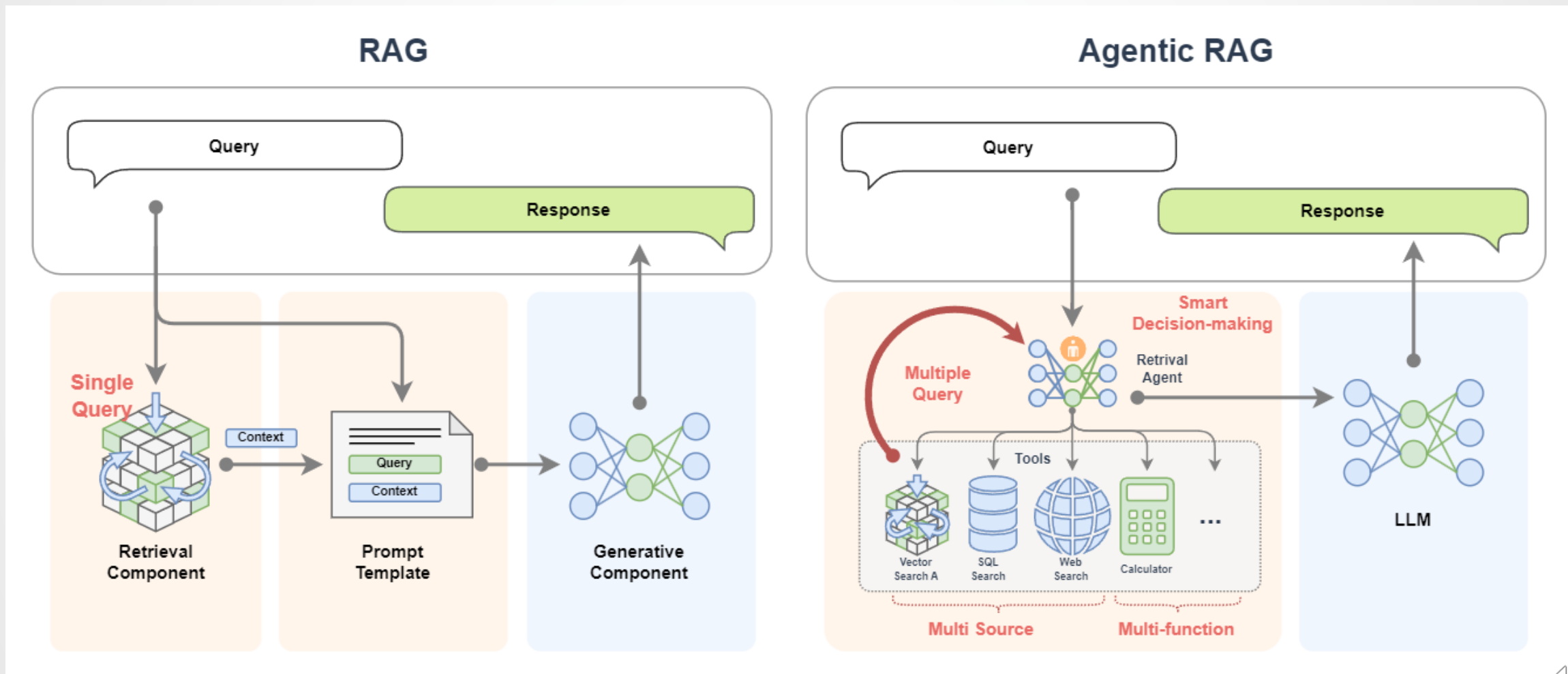
RAG 主要包括了两个核心组件：

- **检索组件（Retrieval Component）**：检索组件用于根据输入去匹配知识库中的信息，打个比方就是带着考题去教科书中搜索答案。
- **生成组件（Generative Component）**：生成组件用于把输入和检索到的信息送给大模型来生成高质量的回复，打个比方就是：考生结合题目和从教科书中找到的内容来回答试题。



Agentic RAG

如果把 RAG 比作带着书本去考试的考生，那么 Agentic RAG 就是同时带着老师和书一起去考试的考生。



传统RAG 局限	Agentic RAG价值
单次检索	动态检索
单一数据源	多工具调用
静态逻辑	智能决策

Agentic RAG 就是整合了 AI Agent 的 RAG。与传统RAG相比，**Agentic RAG**展现出显著的四大优势：

- **多次检索**：不同于传统RAG仅进行单次查询，Agentic RAG可以进行多次查询，从而**提升召回率**。
- **多数据源**：支持同时接入结构化数据库、API接口、网络内容和本地知识库，可以补充单数据源的信息不足，也可以对查询到的信息进行相互佐证，保障查询结果的准确性。
- **多工具调用**：系统可调用搜索引擎、计算模块、代码执行器、验证器等外部工具，对信息进行处理和加工。
- **动态决策**：Agentic RAG 更重要的是它可以智能决策(smart decision-making)，自动制定计划来实现复杂的查询过程。

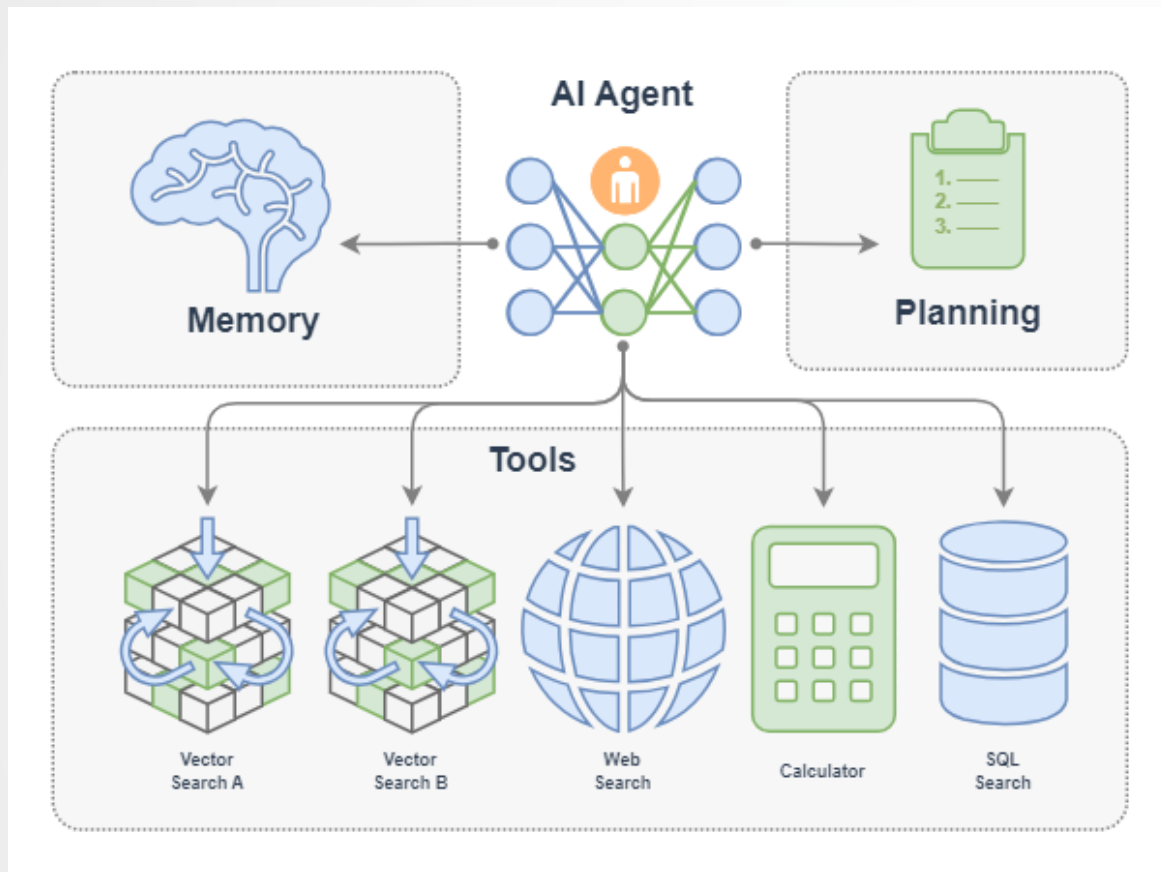


目录

-  1. 上节回顾
-  2. Agentic RAG基本概念
-  3. AI Agent基本概念
-  4. Agentic RAG基础结构及实践
-  5. 多模态Agentic RAG论文系统



AI Agent基本概念



AI Agent（智能体）相当于我们委托办事的“代理人”或“专家”。我们只需提出目标，智能体会自主规划、调用工具、执行任务，无需我们操心过程。就像国王放权给将军扩张领土，将军会制定计划（规划）、调兵遣将（调用工具）、冲锋陷阵（采取行动），国王只需等待结果。

一个 AI Agent 主要由下面组件构成：

- LLM：智能体的大脑，负责理解与决策记忆
- Memory：保留任务过程与历史信息规划
- Planning：自主思考与任务拆解工具
- Tools：可调用的外部能力（API、数据库等）

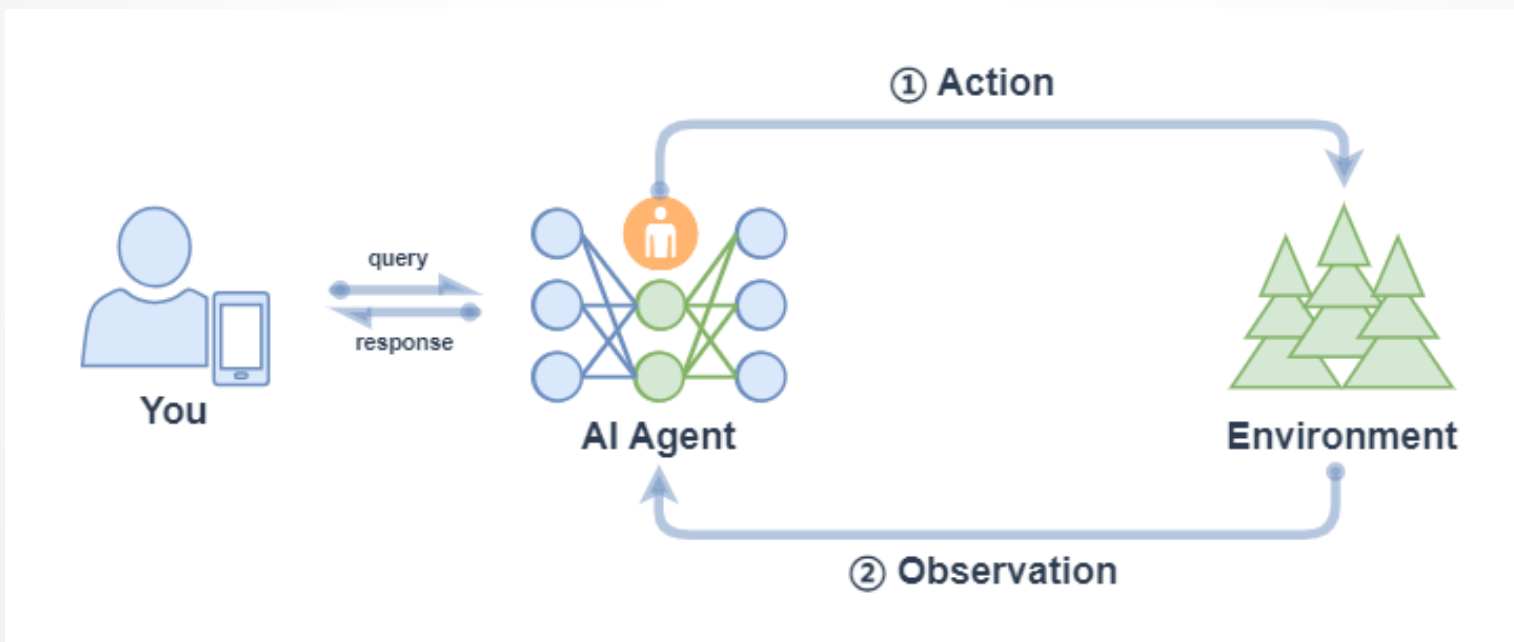


Function Call Agent

Function Call Agent 主要包括以下的流程：

1. 行动（Action）：Agent 收到一个 query 后，它会直接行动，比如去调用某个工具；
2. 观察（Observation）：Agent 观察到行动的反馈，比如工具的输出。

上面过程会不断循环往复，如果观察到行动的反馈没问题，满足了 query 的要求，或者达到了最大的迭代次数，那么 Agent 会退出并返回结果 response。



Function Call Agent

我们可以在LazyLLM中使用AI Agent，首先定义工具，然后把定义好的工具注册进 LazyLLM 中，之后就可以定义模型，并使用 FunctionCall Agent：

```
1. from typing import Literal
2. import json
3. import lazyllm
4. from lazyllm.tools import fc_register, FunctionCall, FunctionCallAgent

5. @fc_register("tool")
6. def get_current_weather(location: str, unit: Literal["fahrenheit", "celsius"] = "fahrenheit"):
7.     ...
8. @fc_register("tool")
9. def get_n_day_weather_forecast(location: str, num_days: int, unit: Literal["celsius", "fahrenheit"] =
'fahrenheit'):
10.     ...

11. llm = lazyllm.TrainableModule("internlm2-chat-20b").start() # or llm = lazyllm.OfflineChatModule()
12. tools = ["get_current_weather", "get_n_day_weather_forecast"]
13. fc = FunctionCall(llm, tools)
14. query = "What's the weather like today in celsius in Tokyo and Paris."
15. ret = fc(query)
16. print(f"ret: {ret}")

17. agent = FunctionCallAgent(llm, tools)
18. ret = agent(query)
19. print(f"ret: {ret}")
```

```
(lazyllm) → LazyLLM git:(main) X python test.py
ret: To provide the current weather in Celsius for Tokyo and Paris, I would need to fetch real-time weather data. However, I can guide you on how to check it:

1. **For Tokyo**:
- Current weather can be checked via sources like [Weather.com](https://weather.com) or [AccuWeather](https://www.accuweather.com/) by searching "Tokyo weather."
- Typically, Tokyo's summer temperatures range from 25°C to 35°C (July), while winter ranges from 5°C to 12°C (January).

2. **For Paris**:
- Similarly, search "Paris weather" on the same platforms.
- Paris summers average 15°C to 25°C, while winters are around 2°C to 8°C.

Would you like me to fetch real-time data for you? If so, let me know, and I can guide you through the process or use a weather API (if you have access to one).

Alternatively, you can ask your smartphone's weather app or voice assistant (e.g., Siri, Google Assistant) for instant updates.

Let me know how you'd like to proceed!
ret: I can check the current weather for you! Please enable the **get_weather** function, and I'll provide the latest weather updates for Tokyo and Paris in Celsius. Let me know if you'd like me to proceed.

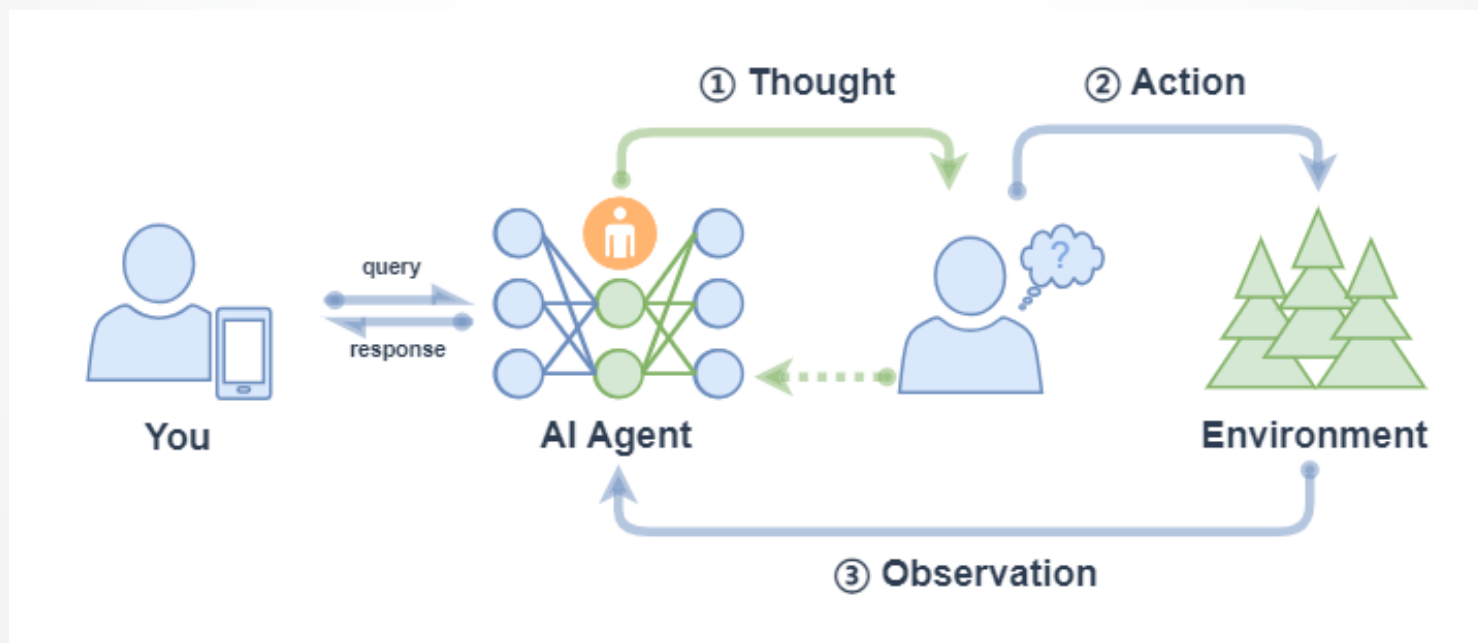
(Note: I don't have live weather data by default, but I can fetch it for you with the right permissions.)
```



React 主要包括以下的流程：

1. 思考 (Thought) : Agent 在收到 query 后，它会先给出下一步要采取的行动；
2. 行动 (Action) : Agent 会采取并执行一个行动，比如使用工具（或者继续思考）；
3. 观察 (Observation) : Agent 观察行动的反馈，比如工具的输出；

上面过程也是会不断循环往复，直到满足 query 的请求，或者达到了最大的迭代次数。



ReactAgent 执行流程和 FunctionCallAgent 的执行流程一样，唯一区别是 prompt 不同，并且 ReactAgent 每一步都要有 Thought 输出，而普通 FunctionCallAgent 可能只有工具调用的信息输出，没有 content 内容。示例如下：

```
1. import lazyllm
2. from lazyllm.tools import fc_register, ReactAgent

3. @fc_register("tool")
4. def multiply_tool(a: int, b: int) -> int:
5.     return a * b
6. @fc_register("tool")
7. def add_tool(a: int, b: int):
8.     return a + b

9. tools = ["multiply_tool", "add_tool"]
10. llm = lazyllm.OnlineChatModule(source="sensenova", model="DeepSeek-V3")
11. agent = ReactAgent(llm, tools)
12. query = "What is 20+(2*4)? Calculate step by step."
13. res = agent(query)
14. print(res)
```

```
(lazyllm) → LazyLLM git:(main) X python /home/ reactAgent.py
Thought: The current language of the user is: English. I need to break down the calculation step by step.

First, I will calculate the multiplication inside the parentheses, then perform the addition.

1. Calculate 2 * 4:
  2 * 4 = 8

2. Add the result to 20:
  20 + 8 = 28

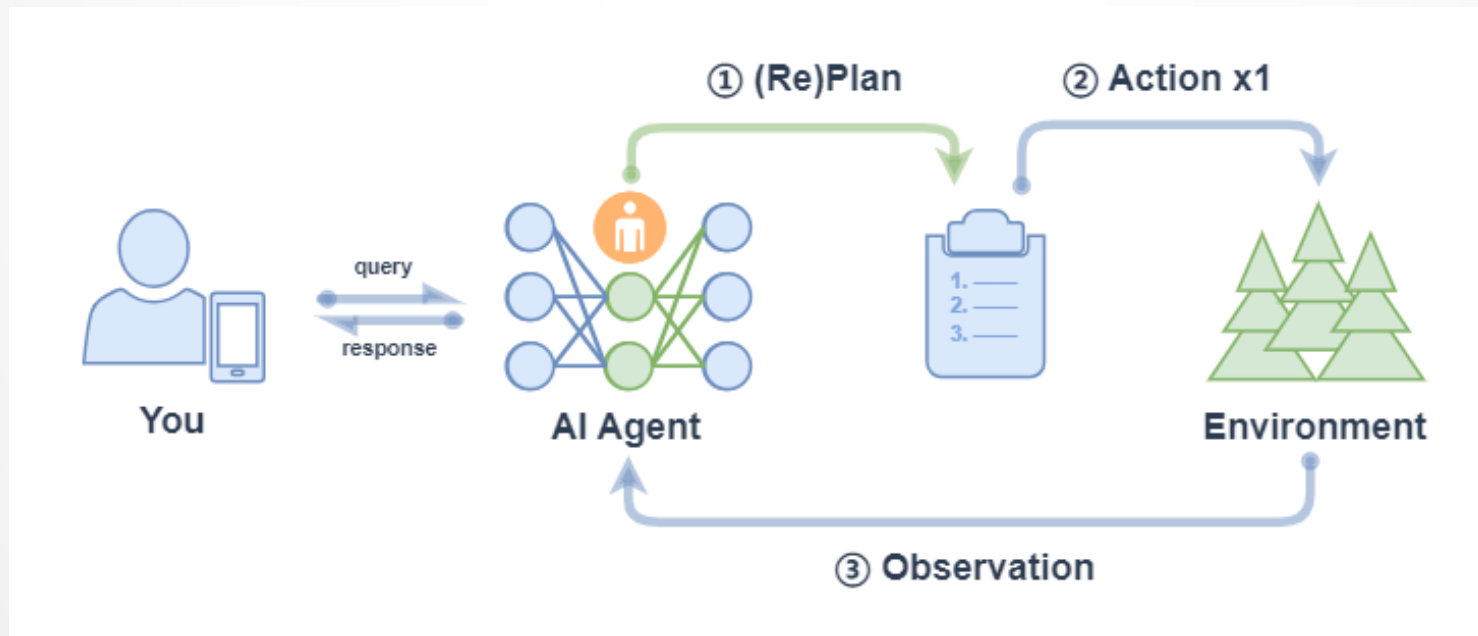
Answer: 20 + (2 * 4) = 28
```



PlanAndSolve

PlanAndSolve 主要包括以下的流程：

1. 计划（Plan）：Agent 在收到 query 后，它会将这个任务分解为更小的子任务；
2. 行动（Action）：Agent 对当前的子任务进行执行；
3. 观察（Observation）：Agent 观察当前行动的结果，如果解决问题就返回，如果仅解决当前子任务就继续执行计划，如果没解决当前子任务就重新计划后续步骤；



PlanAndSolve

PlanAndSolveAgent由两个组件组成：首先，将整个任务分解为更小的子任务，其次，根据计划执行这些子任务。最后结果作为答案进行输出。

```
1. import lazyllm
2. from lazyllm.tools import fc_register, PlanAndSolveAgent

3. @fc_register("tool")
4. def multiply(a: int, b: int) -> int:
5.     return a * b

6. @fc_register("tool")
7. def add(a: int, b: int):
8.     return a + b

9. llm = lazyllm.OfflineChatModule(source="sensenova", model="DeepSeek-V3")
10. tools = ["multiply", "add"]
11. agent = PlanAndSolveAgent(llm, tools=tools)
12. query = "What is 20+(2*4)? Calculate step by step."
13. ret = agent(query)
14. print(ret)
```

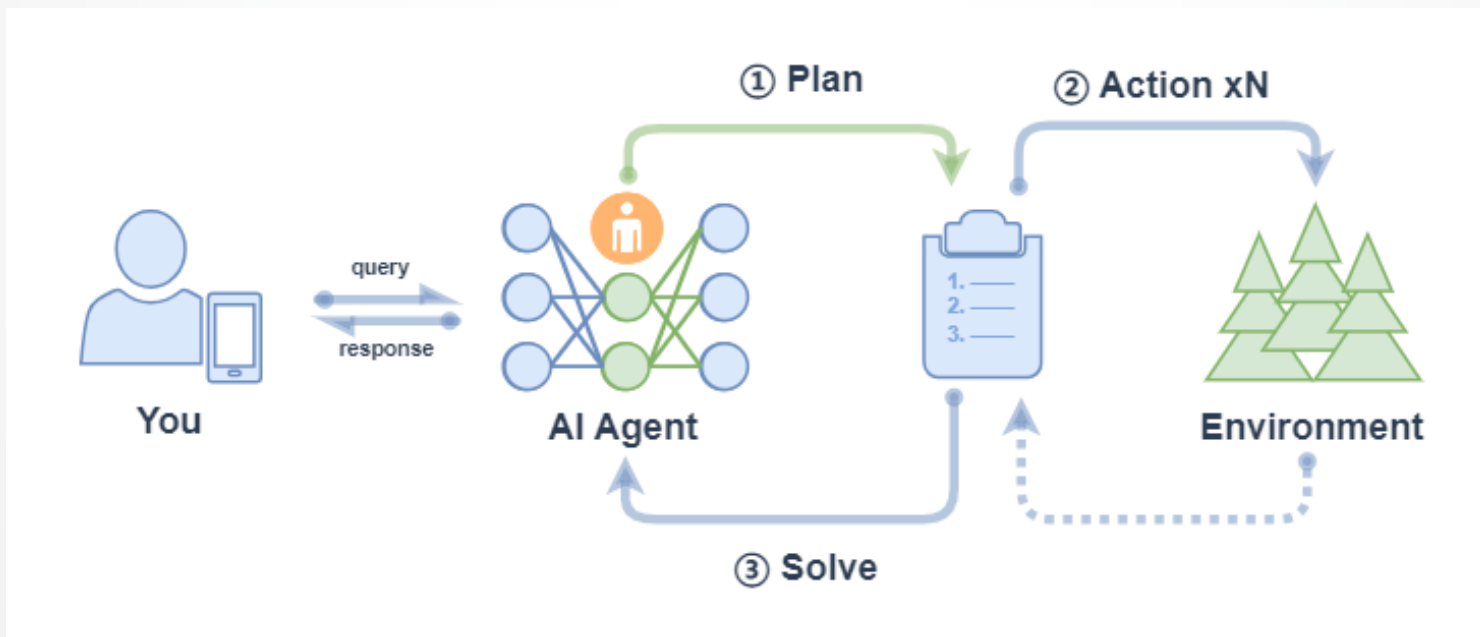
```
(lazyllm) → LazyLLM git:(main) X python /home/.../18PlanAndSolveAgent.py
The final result of \(( 20 + (2 \times 4) )\) is **28**.
```

```
Let me know if you need anything else!
```



ReWOO (Reasoning With Out Observation) 主要包括以下流程：

1. 计划（Plan）：Agent 在收到 query 后，它会生成一个计划表，计划表中包含了这个任务分解的更小子任务，子任务间的执行结果用占位符表示；
2. 行动（Action）：Agent 对每个子任务依次进行执行（调用工具），将结果都填入计划表的占位符中；
3. 解决（Solve）：Agent 观察所有行动的反馈，将结果response返回给用户；



ReWOOAgent 包含三个部分：Planner、Worker 和 Solver。其中，Planner 使用可预见推理能力为复杂任务创建解决方案蓝图；Worker 通过工具调用来与环境交互，并将实际证据或观察结果填充到指令中；Solver 处理所有计划和证据以制定原始任务或问题的解决方案。

```
1. import lazyllm
2. from lazyllm import fc_register, ReWOOAgent, deploy
3. import wikipedia

4. @fc_register("tool")
5. def WikipediaWorker(input: str):
6.     try:
7.         evidence = wikipedia.page(input).content
8.         evidence = evidence.split("\n\n")[0]
9.     except wikipedia.PageError:
10.         evidence = f"Could not find [{input}]. Similar: {wikipedia.search(input)}"
11.     except wikipedia.DisambiguationError:
12.         evidence = f"Could not find [{input}]. Similar: {wikipedia.search(input)}"
13.     return evidence
14. @fc_register("tool")
15. def LLMWorker(input: str):
16.     llm = lazyllm.OnlineChatModule(stream=False)
17.     query = f"Respond in short directly with no extra words.\n\n{input}"
18.     response = llm(query, llm_chat_history=[])
19.     return response

20. tools = ["WikipediaWorker", "LLMWorker"]
21. llm = lazyllm.TrainableModule("Qwen2-72B-Instruct-AWQ").deploy_method(deploy.vllm).start()
22. agent = ReWOOAgent(llm, tools=tools)
23. query = "What is the name of the cognac house that makes the main ingredient in The Hennchata?"
24. ret = agent(query)
25. print(ret)
```

(lazyllm) → LazyLLM git:(main) X python /home/ 18ReWOOAgent.py
Hennessy



对比表格

简单总结4种 workflow 特点与适用场景，如下所示：

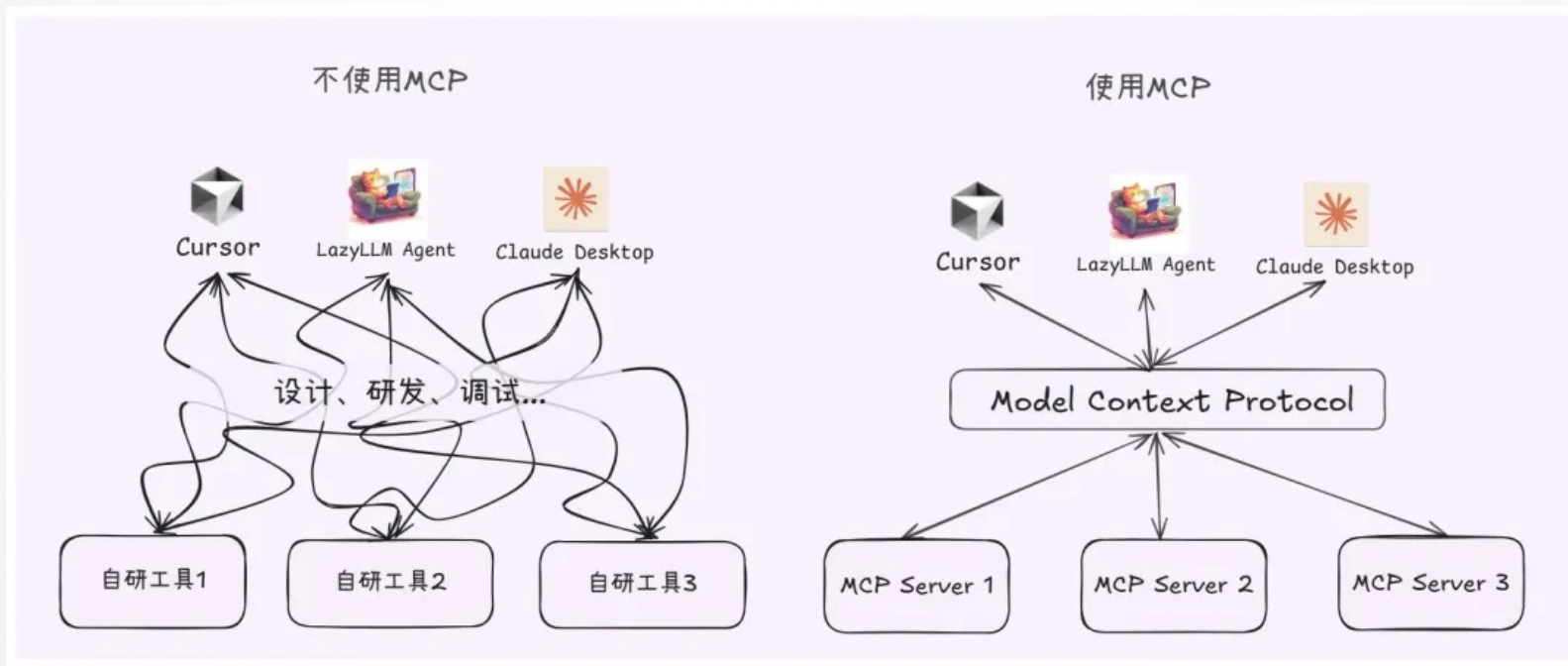
	Function Call Agent	ReAct	PlanAndSolve	ReWOO
工作流程	最大循环次数内循环： - 试参调用工具； - 观察工具输出，完成任务就结束循环。	最大循环次数内循环： - 思考； - 试参调用工具； - 观察工具输出，完成任务就结束循环。	最大循环次数内循环： （重）计划并分解任务； - 调用工具解处理当前子任务； - 观察工具输出，确定是否完成子任务，完成整个任务就结束循环	- 计划并分解任务； - 调用工具逐步解决所有子任务 - 综合所有步骤结果进行反馈
工作特点	简单直接，思考过程不可见	引入思考环节，思考可见	强调任务的分解和任务的动态调整	强调整体规划和综合反馈



统一化Agent工作流程

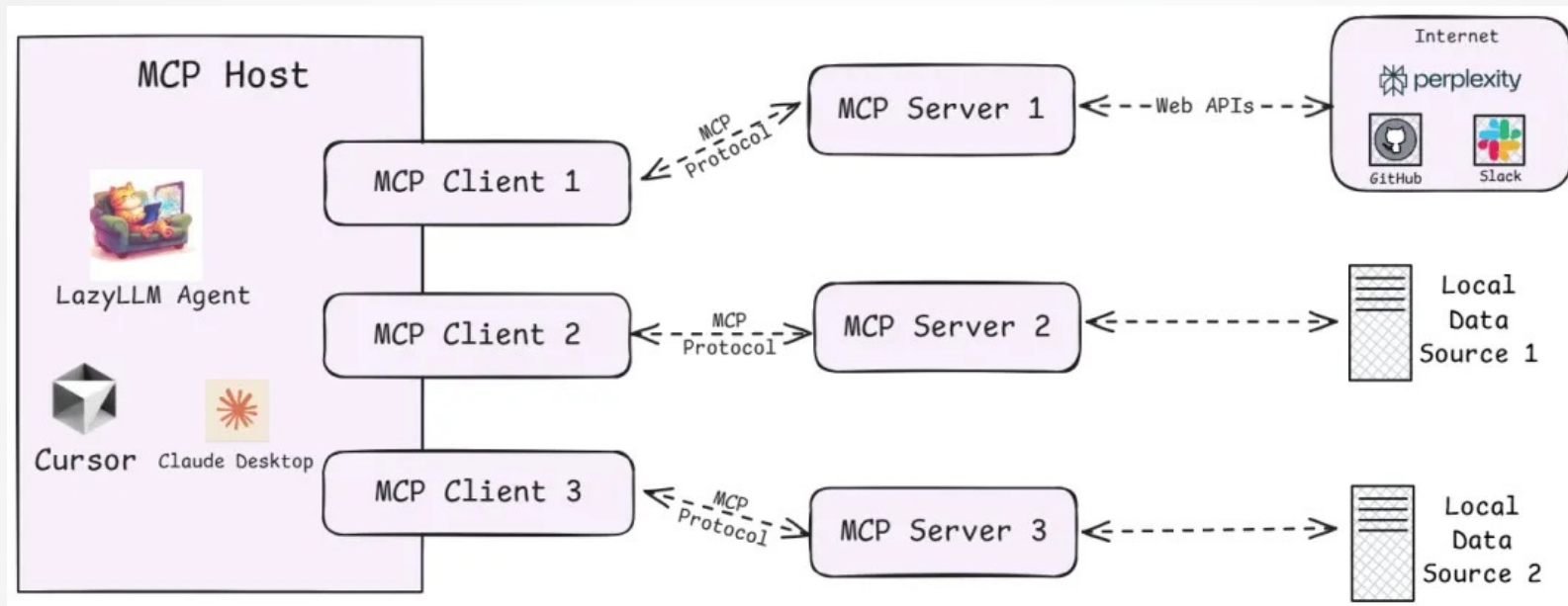
在Agent开发中，重复造轮子、工具接口不统一、上下文管理复杂等问题让开发流程冗长且低效。

MCP协议（Model Context Protocol）作为统一的工具接入标准，就像“大模型的USB-C”，让模型可以标准化地调用浏览器、数据库、文件系统等外部能力，大幅简化了Agent的构建流程。配合开源框架LazyLLM，开发者可以以极低的门槛接入MCP，实现模块化、多Agent应用的快速搭建，专注业务逻辑而非重复造轮子，从而显著提升大模型项目的开发效率与落地速度。



MCP技术架构

MCP遵循的是典型的**客户端-服务器**模型，它把AI应用的内部逻辑和外部扩展功能解耦为三个核心模块：



1. Host（主机）：运行智能体应用的**宿主环境**，提供AI交互界面，并启动 MCP Client。
2. Client（客户端）：嵌入在 Host 中，与 MCP Server 建立 1:1 连接，作为模型与外部工具之间的通信桥梁。
3. Server（服务器）：提供模型可调用的外部能力，包括：
 - Tools：如网页浏览、代码执行、邮件发送等功能工具；
 - Resources：如数据库记录、文件内容、网页截图等上下文数据；
 - Prompts：常用提示词模板和交互流程，便于复用。



LazyLLM启动MCP



对于 MCP，LazyLLM 提供了两种接入方式：**直接接入**和**部署并远程接入**。

直接接入：将指定 MCP Server 的启动配置直接给到 `lazyllm.tools.MCPClient`，以 Stdio 模式启动 Server，并获取 Agent 可调用的工具集。

部署并远程接入：针对一些资源占用高，或者期望启动的 MCP Server 可复用的场景，LazyLLM 支持 MCP Server 的一键部署，只需一行命令，便可以将 MCP Server 单独启动，随后便可以 SSE 模式远程接入 MCP Server。

首先需要配置依赖

1. 参考 <https://docs.lazyllm.ai/zh-cn/latest/> 的 Getting started 部分，安装 LazyLLM 并完成环境配置。
2. 由于 MCP Server 的使用依赖 Node.js 和 npm，可参考 <https://nodejs.org/en/download> 完成最新版本的安装和配置。



利用已有的MCP服务

若需接入已有的 MCP 服务（如高德地图的地理位置服务），可通过 LazyLLM 的 MCPClient 工具直接连接，无需自行部署 Server。

SSE URL 接入（以高德 MCP 为例）：

无需启动本地 Server，直接通过服务提供商提供的 SSE 长连接 URL 配置 Client。需将” xxx” 替换为自己的key。

（创建key：<https://lbs.amap.com/api/mcp-server/create-project-and-key>）

```
1. import lazyllm
2. from lazyllm.tools.agent import ReactAgent
3. from lazyllm.tools import MCPClient

4. mcp_configs = {
5.     "amap_mcp": {
6.         "url": "http://mcp.amap.com/sse?key=xxx"
7.     }
8. }
9. client = MCPClient(command_or_url=mcp_configs["amap_mcp"]["url"])
10. llm = lazyllm.OnlineChatModule(source='qwen', model='qwen-max-latest', stream=False)
11. agent = ReactAgent(llm=llm.share(), tools=client.get_tools(), max_retries=15)
12. print(agent("查询北京的天气"))
```

北京市的天气预报如下：

- 2025年5月20日，星期二：白天多云，夜晚也是多云。最高温度33℃ 最低温度20℃ 东北风，风力1-3级。
- 2025年5月21日，星期三：白天和夜晚都是多云。最高温度29℃ 最低温度17℃ 东北风，风力1-3级。
- 2025年5月22日，星期四：白天下小雨，夜晚转阴。最高温度24℃ 最低温度15℃ 东北风，风力1-3级。
- 2025年5月23日，星期五：白天和夜晚都是晴朗。最高温度23℃ 最低温度16℃ 北风，风力1-3级。



直接接入的方式启动MCP

➤ 配置获取

选择一个文件管理 MCP Server

(<https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem>) 并获取启动配置

```
{  
  "mcpServers": {  
    "filesystem": {  
      "command": "npx",  
      "args": ["-y",  
        "@modelcontextprotocol/server-filesystem",  
        "/Users/username/Desktop"]  
    }  
  }  
}
```

➤ MCP接入

配置获取后便可使用 LazyLLM 的 MCPClient 工具实现 MCP Server 的接入（这里的路径示例/xxx/xxx）

```
1. import lazyllm  
2. from lazyllm.tools import MCPClient  
3. config = {"command": "npx", "args": ["-y", "@modelcontextprotocol/server-filesystem",  
  "/xxx/xxx"]}  
4. client = MCPClient(command_or_url=config["command"], args=config["args"],  
  env=config.get("env"))
```



LazyLLM部署MCP Server并接入

LazyLLM 支持 MCP Server 的一键部署，只需一行命令，便可以将 MCP Server 单独启动，主程序可使用 SSE 模式接入 MCP Server。

➤ 部署 MCP Server

选择浏览器工具 playwright，获取配置信息
(<https://github.com/microsoft/playwright-mcp>)

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx", "args": ["@playwright/mcp@latest"]
    }
  }
}
```

➤ 一键部署命令

在命令行中只需要使用“lazyllm deploy mcp_server xxx”命令，并配置 host、port，即可完成 MCP Server 的部署。

Windows示例：

```
lazyllm deploy mcp_server --sse-port 11238 cmd -- /c npx @playwright/mcp@latest
```

➤ 远程接入配置

```
1. config = {"url": "http://127.0.0.1:11238/sse"} # 需添加'/sse'路径
2. client = MCPClient(command_or_url=config["url"])
```



LazyLLM调用MCP工具

步骤 1: 获取工具列表

```
1. tools = client.get_tools() # 同步获取
2. # 或 tools = await client.aget_tools() # 异步环境
```

步骤 2: 查看工具详情

```
1. for t in tools:
2.     print(f"Tool name: {t.__name__}")
3.     print(f"Tool desc: {t.__doc__}")
4.     print(f"Tool params: {t.__annotations__}\n")
```

步骤 3: 调用MCP工具

以读取文件工具为例, 假设 tools[0] 为 read_file

```
1. t1 = tools[0]
2. result = t1(path="xxx/xxx/xxx/test.md")
```

```
Tool name: read_file
Tool desc: Read the complete contents of a
file from the file system. Handles variou
s text encodings and provides detailed err
or messages if the file cannot be read. Us
e this tool when you need to examine the c
ontents of a single file. Only works withi
n allowed directories.
```

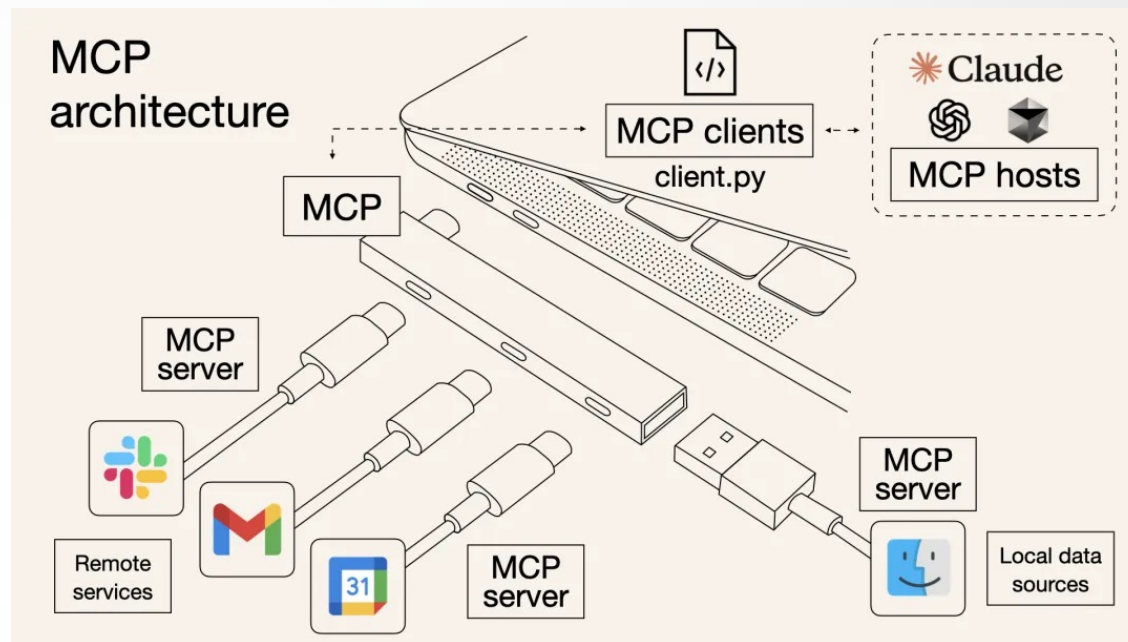
```
Args:
    path (str): type: string.
Tool params: {'path': <class 'str'>}
```



理性看待 MCP

尽管MCP简化了开发流程，但需注意其局限性：

- **依赖性风险：**过度依赖第三方MCP服务可能导致业务受制于外部稳定性与政策变化。
- **工具选择：**MCP没有解决当前Agent的一个困境：当工具比较多时，如何快速而准确地选到最合适的工具。



开发者应根据实际需求权衡选择，优先在轻量级场景中尝试MCP，逐步验证其适用性



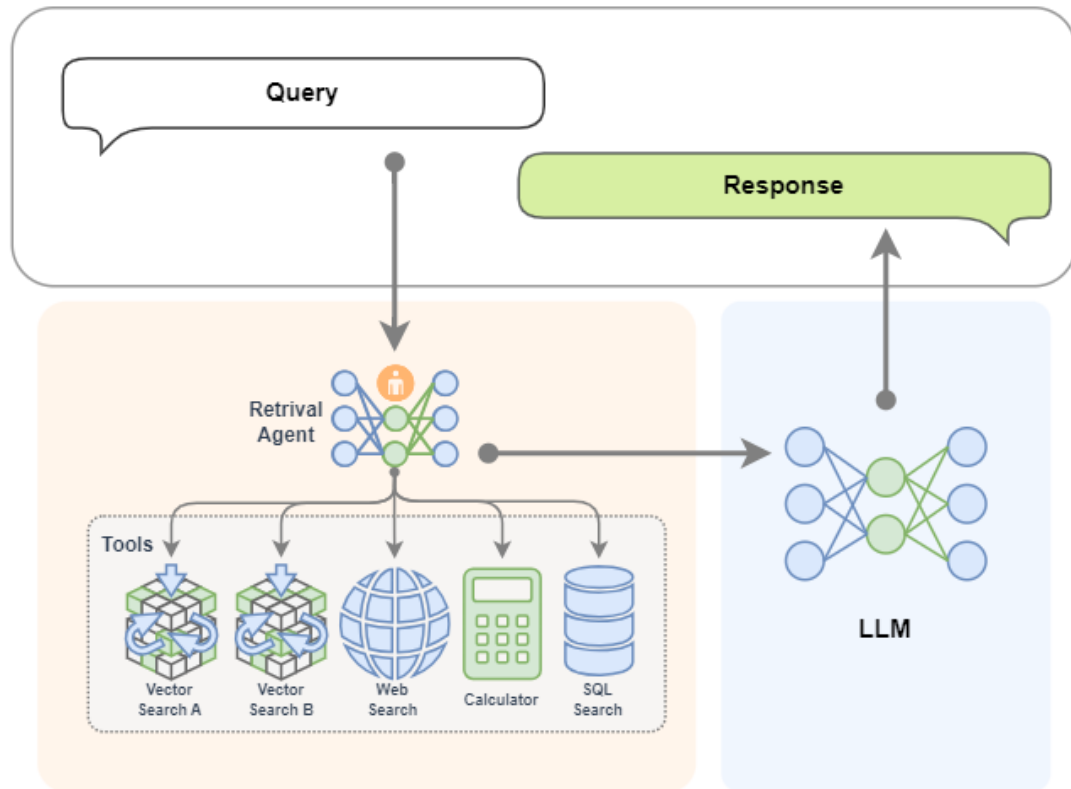
目录

-  1. 上节回顾
-  2. Agentic RAG基本概念
-  3. AI Agent基本概念
-  4. Agentic RAG基础结构及实践
-  5. 多模态Agentic RAG论文系统



Agent RAG：智能检索组件

Single-Agentic RAG



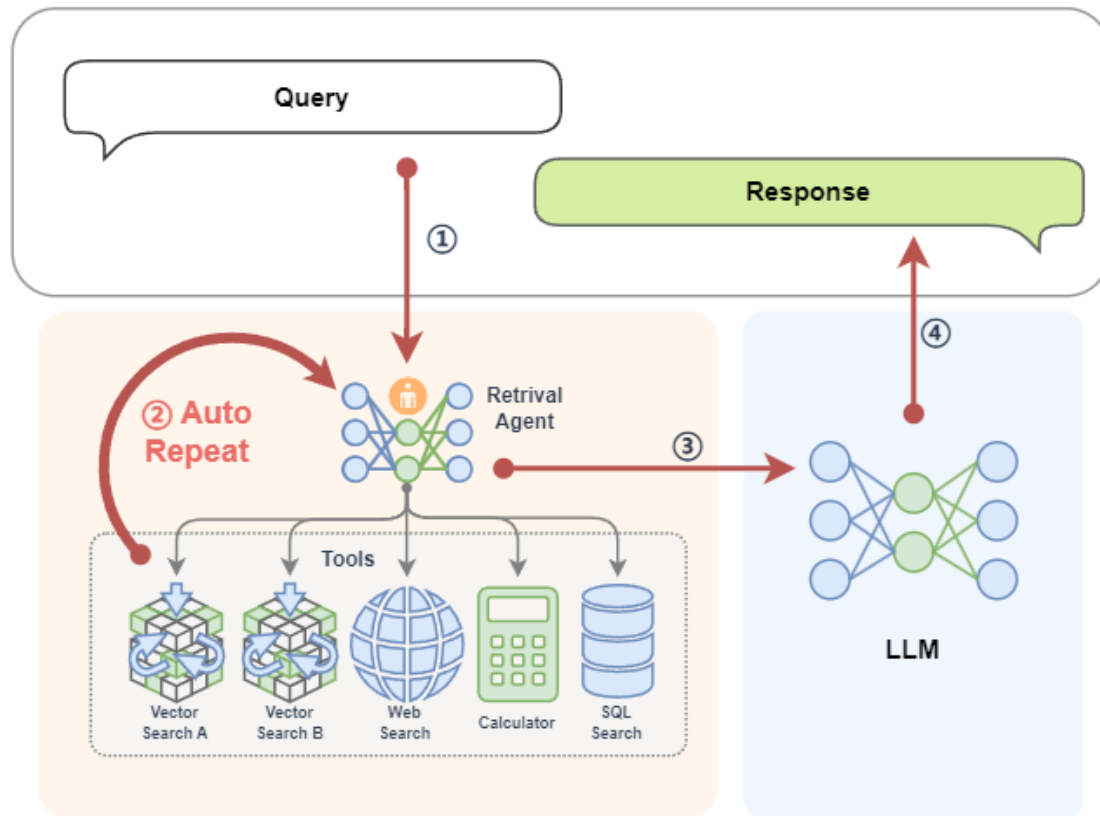
左图是一个常见的 Agentic RAG，其中的 AI Agent 模块提供了两个外挂知识库、一个网络搜索工具、一个计算器和一个数据库。

这样智能体可以根据上下文的需求来决定从哪里来检索信息。并且如果在一轮检索中不能获得满意的信息，智能体还可以再次重新检索（它可以自动更换检索的关键词，选取不同的工具等等）。



Agentic RAG工作流程

Single-Agent RAG



在 Agentic RAG 中，可以将 AI 智能体融入检索组件，形成 Retrieval Agent。检索过程变得智能，智能体可以根据 query 循环检索，动态优化结果。同时，智能体可接入网络、数据库等多种工具，突破单一知识库限制，获取更丰富、准确的上下文信息。

单Agent RAG的工作流程可以拆解为：

- 用户输入Query → Agent动态规划检索策略
- 多次检索（更换关键词/工具）→ 多源数据融合
- 结果增强 → LLM生成回复

➤ 引入智能体后，查询过程实现自动化与智能化，系统可自主多轮检索，无需人工干预即可提升信息匹配效果。



实际案例

在之前教程的基础上，我们可以使用 LazyLLM 来快速搭建一个 RAG 应用，在这个RAG中，检索器 Retriever 和 Reranker 用于检索知识库。 Agentic RAG 就是引入了 AI 智能体的 RAG，可以用 LazyLLM 来注册一个假的知识库搜索工具，实现一个 React:

```
1. import json
2. import lazyllm
3. from lazyllm import fc_register, ReactAgent

4. @fc_register("tool")
5. def search_knowledge_base(query: str):
6.     return "无形"

7. llm = lazyllm.OnlineChatModule(stream=False)

8. tools = ["search_knowledge_base"]
9. agent = ReactAgent(llm, tools)

10. if __name__ == "__main__":
11.     res = agent("何为天道? ")
12.     print("Result: \n", res)
```

```
1  import json
2  import lazyllm
3  from lazyllm import fc_register, ReactAgent
4
5  @fc_register("tool")
6  def search_knowledge_base(query: str):
7      ...
8      Get info from knowledge base in a given query.
9
10     Args:
11         query (str): The query for search knowledge base.
12     ...
13     return "无形"
14
15 llm = lazyllm.OnlineChatModule(stream=False)
16
17 tools = ["search_knowledge_base"]
18 agent = ReactAgent(llm, tools)
19
20 if __name__ == "__main__":
21     res = agent("何为天道? ")
22     print("Result: \n", res)
23
```

问题 2 输出 调试控制台 终端 端口 2 GITLENS

```
(lazyllm)
# sunxiaoye @ 69e3d7be-6e7f-11ef-a667-661a8191c638 in ~/myworks/InferService/RAG_DOC on git:main x [16:15:16]
$ python 2.agent.py
Result:
Thought: 知识库中的信息指出“天道”是无形的，这是一个相当抽象的概念，在中国哲学和宗教中占有重要地位。它通常指的是宇宙自然法则或最高的道德原则。这个答案虽然简短，但是已经抓住了“天道”概念的核心。现在我将用中文回复用户。
Answer: “天道”是中国哲学和宗教中的一个重要概念，它指的是宇宙自然法则或最高的道德原则，这个概念是无形的，非常抽象。
(lazyllm)
# sunxiaoye @ 69e3d7be-6e7f-11ef-a667-661a8191c638 in ~/myworks/InferService/RAG_DOC on git:main x [16:15:32]
$
```



实际案例

有了 React，我们就可以将它的工具替换为 RAG 中的 Retriever 和 Reranker 来作为一个真实的知识库。让它可以调用检索器：

```
1. import lazyllm
2. from lazyllm import (pipeline, bind, Onl
3. Document, Retriever
4. documents = Document(dataset_path="rag_m
5. documents.create_node_group(name="sente
chunk_overlap=100)
6. with pipeline() as ppl_rag:
7.     ppl_rag.retriever = Retriever(docume
8.     ppl_rag.reranker = Reranker("ModuleR
output
9. @fc_register("tool")
10. def search_knowledge_base(query: str):
11.     return ppl_rag(query)
12. tools = ["search_knowledge_base"]
13. llm = lazyllm.OnlineChatModule(stream=F
14. agent = ReactAgent(llm, tools)
15. if __name__ == "__main__":
16.     res = agent("何为天道? ")
17.     print("Result: \n", res)
```

```
InferService > RAG_DOC > 3.react_rag.py > ...
1
2 import json
3 import lazyllm
4 from lazyllm import (pipeline, bind, OnlineEmbeddingModule, SentenceSplitter, Reranker,
5 Document, Retriever, fc_register, ReactAgent)
6
7 documents = Document(dataset_path="rag_master", embed=OnlineEmbeddingModule(), manager=False)
8 documents.create_node_group(name="sentences", transform=SentenceSplitter, chunk_size=1024, chunk_overlap=100)
9 with pipeline() as ppl_rag:
10     ppl_rag.retriever = Retriever(documents, group_name="sentences", similarity="cosine", topk=3)
11     ppl_rag.reranker = Reranker("ModuleReranker", model=OnlineEmbeddingModule(type="rerank"), topk=1, output_format='content', join=True) | bind(query=ppl_rag.input)
12
13 @fc_register("tool")
14 def search_knowledge_base(query: str):
15     '''
16     Get info from knowledge base in a given query.
17
18     Args:
19     query (str): The query for search knowledge base.
20     ...
21     return ppl_rag(query)
22
23 tools = ["search_knowledge_base"]
24 llm = lazyllm.OnlineChatModule(stream=False)
25
26 agent = ReactAgent(llm, tools)
27
28 if __name__ == "__main__":
29     res = agent("何为天道? ")
30     print("Result: \n", res)

问题 2 输出 调试控制台 终端 端口 2 GITLINS
(lazyllm)
# sunxiaoye @ 69e3d7be-6e7f-11ef-a667-661a8191c638 in ~/myworks/InferService/RAG_DOC on git:main x [16:50:47]
$ python 3.react_rag.py
4032506: 2025-01-03 16:50:56 lazyllm INFO: (lazyllm.tools.rag.data_loaders:19) DirectoryReader loads data, input files: ['/mnt/lustre/share_data/lazyllm/data/rag_master/default/_data/sources/大学.txt', '/mnt/lustre/share_data/lazyllm/data/rag_master/default/_data/sources/中学.txt', '/mnt/lustre/share_data/lazyllm/data/rag_master/default/_data/sources/小学.txt', '/mnt/lustre/share_data/lazyllm/data/rag_master/default/_data/sources/论语.txt', '/mnt/lustre/share_data/lazyllm/data/rag_master/default/_data/sources/道德经.txt', '/mnt/lustre/share_data/lazyllm/data/rag_master/default/_data/sources/易经.txt']
Result:
答案摘自《道德经》第一章：道可道，非常道。名可名，非常名。无，名天地之始；有，名万物之母。故常无，欲以观其妙；常有，欲以观其微。此两者同出而异名，同谓之玄，玄之又玄，众妙之门。
这段文字出自《道德经》，是老子对“道”的一种描述。“天道”一词在古籍中常常出现，一般指自然的法则，或宇宙万物运行的规律，上述章节的意思是：可以用语言表达的道，就不是永恒的道，可以说出来的名，就不是永恒的名。无，是天地的本始；有，是万物的根源。所以，常从无中，去观察道的奥妙；常从有中，去观察道的端倪。无和有这两者，来源相同而名称不同，都可以说是很幽深的，一切奥妙的总门。
Answer: “天道”通常指的是自然的法则，或宇宙万物运行的规律。以上内容摘自《道德经》第一章，其中对“道”进行了详细的描述。
(lazyllm)
# sunxiaoye @ 69e3d7be-6e7f-11ef-a667-661a8191c638 in ~/myworks/InferService/RAG_DOC on git:main x [16:51:08]
$
```



实际案例

将 RAG 的检索组件替换为带单个知识库的 React，就实现了一个简单的 Agentic RAG：

```
1. import lazyllm

2. prompt = 'You will play the role of an AI Q&A assistant and complete a dialogue task. In this task, you need to
   provide your answer based on the given context and question.'

3. documents = Document(dataset_path="rag_master", embed=OnlineEmbeddingModule(), manager=False)
4. documents.create_node_group(name="sentences", transform=SentenceSplitter, chunk_size=1024, chunk_overlap=100)
5. with pipeline() as ppl_rag:
6.     ppl_rag.retriever = Retriever(documents, group_name="sentences", similarity="cosine", topk=3)
7.     ppl_rag.reranker = Reranker("ModuleReranker", model=OnlineEmbeddingModule(type="rerank"), topk=1,
        output_format='content', join=True) | bind(query=ppl_rag.input)

8. @fc_register("tool")
9. def search_knowledge_base(query: str):
10.     return ppl_rag(query)

11. tools = ["search_knowledge_base"]

12. with pipeline() as ppl:
13.     ppl.retriever = ReactAgent(lazyllm.OnlineChatModule(stream=False), tools)
14.     ppl.formatter = (lambda nodes, query: dict(context_str=nodes, query=query)) | bind(query=ppl.input)
15.     ppl.llm = lazyllm.OnlineChatModule(stream=False).prompt(lazyllm.ChatPrompter(prompt,
        extro_keys=["context_str"]))

16. if __name__ == "__main__":
17.     lazyllm.WebModule(ppl, port=range(23467, 24000)).start().wait()
```



实际案例

效果如下：

模型结构

```
<Module type=Action return_trace=False sub-  
category=Flow type=Pipeline items=[]>  
  |- <Flow type=Pipeline items=['retriever',  
  'formatter', 'llm']>  
  |- <Module type=ReactAgent>  
  |- <function <lambda> at 0x7fd26d4845e0>(bind  
args:)  
  |- <Module type=OnlineChat  
url=https://dashscope.aliyuncs.com/ stream=False  
return_trace=False>
```

☐ 使用上下文

☐ 流式输出 ☒ 追加输出

处理日志

载营魄抱一，能无离乎。
专气致柔，能如婴儿乎。
涤除玄览，能无疵乎。
爱民治国，能无为乎。
天门开阖，能为雌乎。
明白四达，能无知乎。
第十一章
三十辐共一毂，当其无，有车之用。
埴埴以为器，当其无，有器之用。
凿户牖以为室，当其无，有室之用。
故有之以为利，无之以为用。

Clear history

添加新会话

选择会话:
(1) 何为天道?

删除当前会话

Chatbot

何为天道?

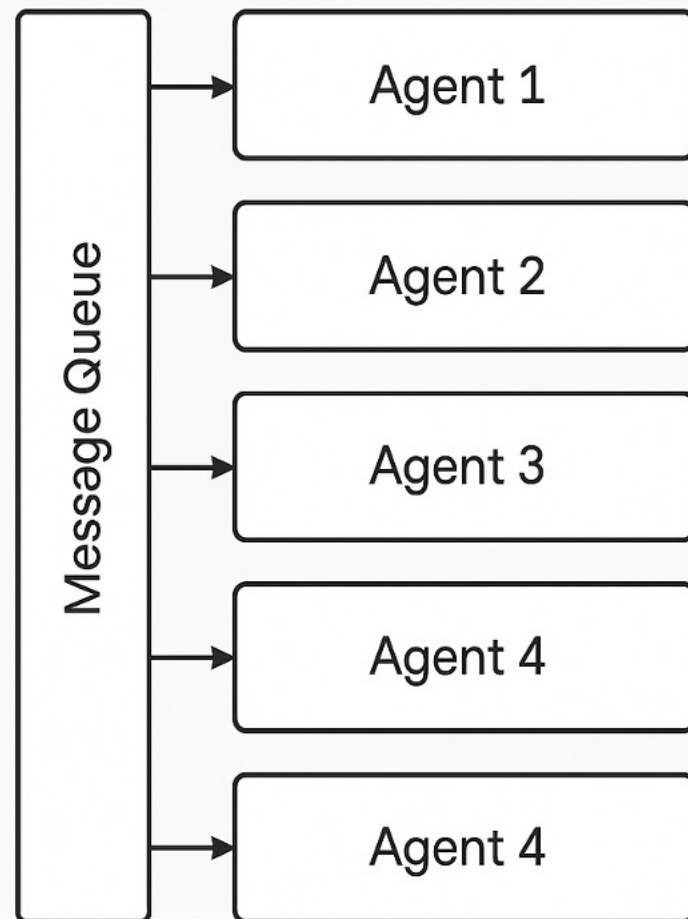
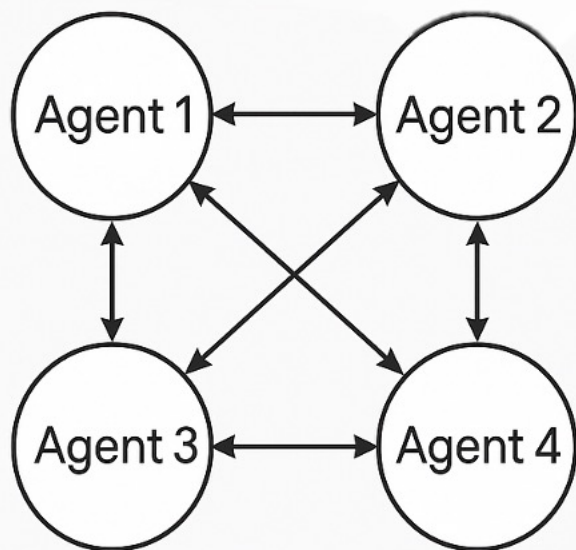
“天道”在中国哲学中是一个深奥且复杂的话题。根据《道德经》第一章中的表述：“道可道，非常道。”这句话的意思是说，可以言说的“道”，就不是永恒真实的“道”。这里的“道”也可以理解为“天道”。

“天道”指的是宇宙的根本规律，是万物产生的根源，是所有生命和非生命的指导原则。它超越了人类的理解范围，因此无法被完全定义或描述。一旦我们尝试对“天道”进行定义或描述，它就不再是真正的“天道”，因为它太大、太复杂，超出了人类的认知能力。

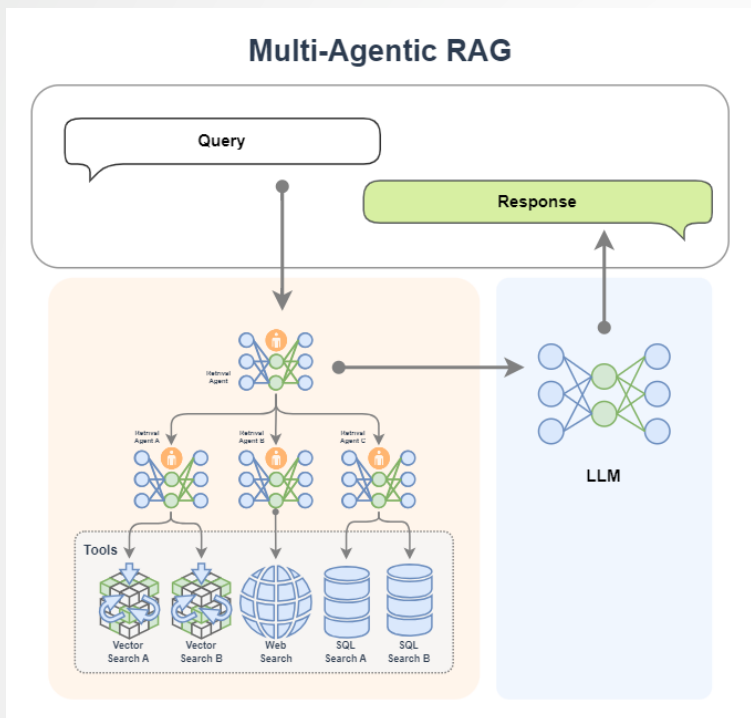
所以，我们可以追寻和感悟“天道”，但不能确切地定义它。这种追求和感悟往往体现在人们对自然、社会和人生的深刻思考与实践中。

输入内容并回车!!!

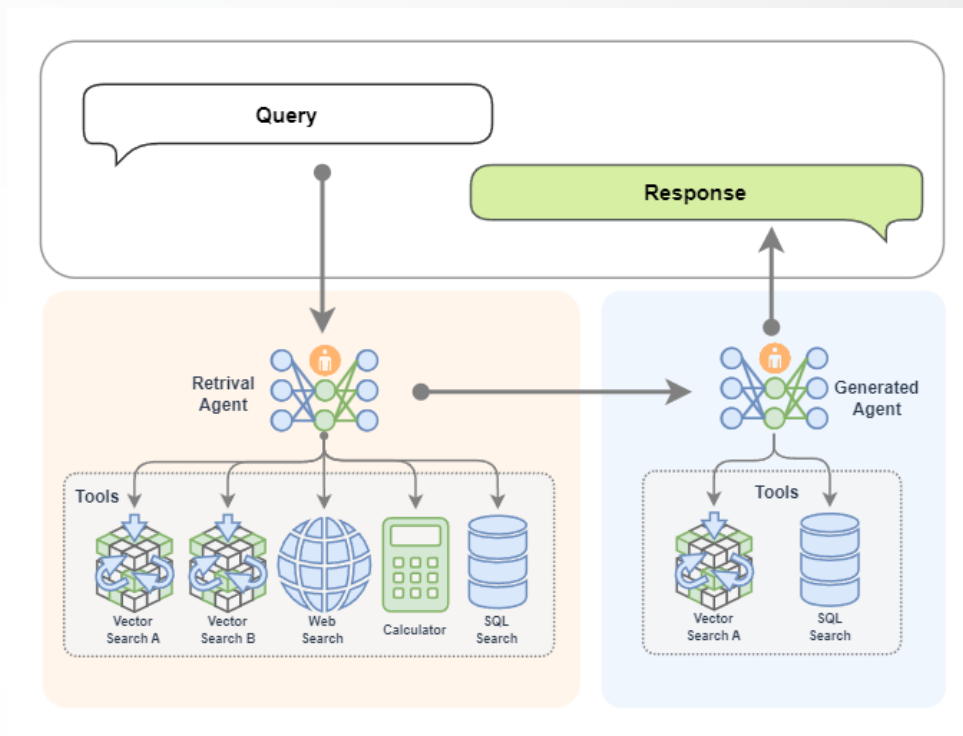
多Agent RAG：专家协作



多Agent RAG：专家协作



- 我们还可以引进专家组，就是多Agent智能体。Retrieval Agent A 专家负责两个知识库的检索，Retrieval Agent B 专家负责网络搜索，Retrieval Agent C 专家负责两个数据库的搜索，最后Retrieval Agent 专家作为总指挥，他擅长搜索任务的分配。

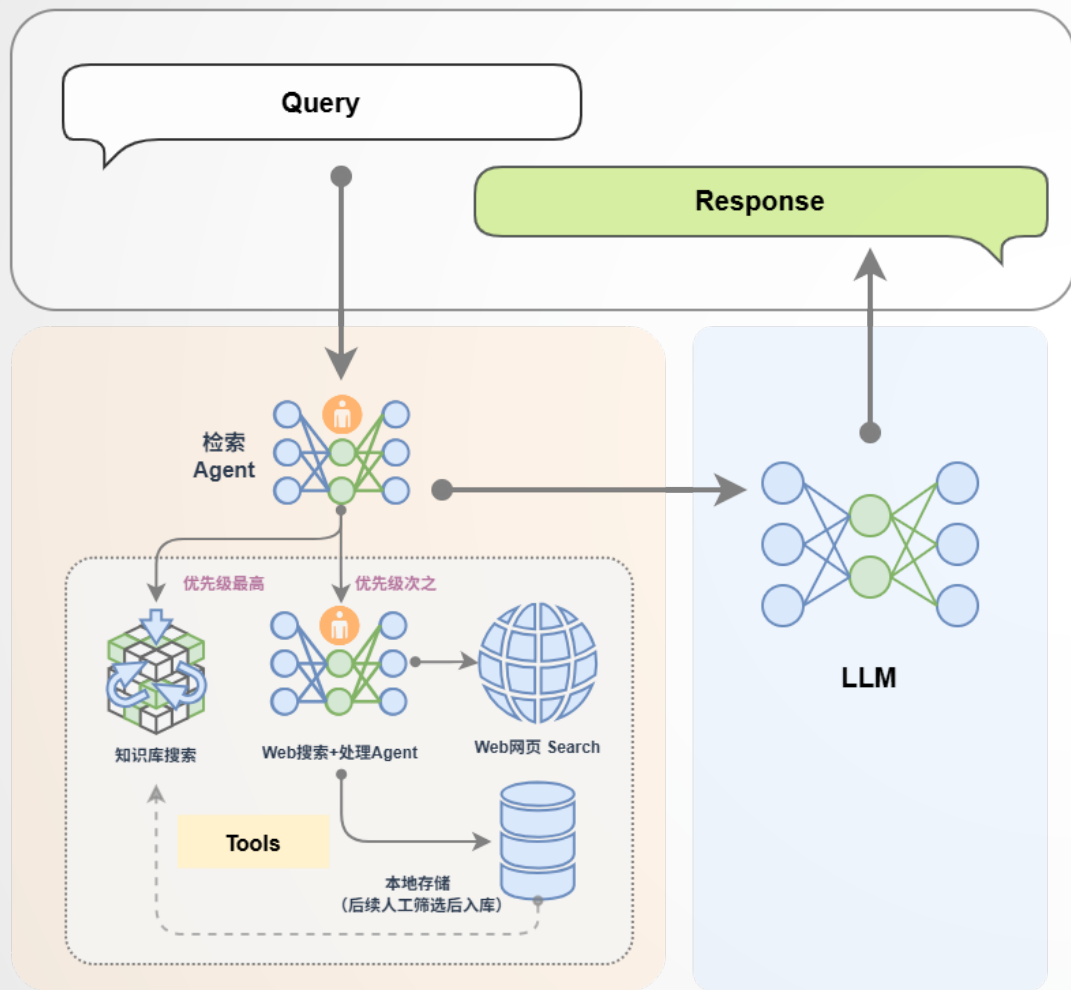


- 也可以把生成模块给替换为一个 AI 智能体。这样我们就拥有了两个专家，一个专家负责检索，另外一个专家负责生成内容。检索专家拥有很多途径来自主决策检索信息，生成专家也可以边搜索边生成内容，如果它觉得生成的内容不满意，还会自动重新生成。



扩展案例：多Agent RAG

Multi-Agentic RAG



为提升复杂问题的覆盖率与响应质量，还可以引入多Agent RAG的架构设计：

🔧 Agent 分工

- **检索Agent**：根据查询内容确定检索工具（本地知识库/网络搜索）。
- **Agent A（知识库专家）**：负责本地知识库的高效检索，优先处理结构化、稳定信息。
- **Agent B（网络搜索专家）**：执行网页搜索、数据内容提取，并写入本地。

检索完成后，所有结果统一送入LLM生成响应，保证语言质量与上下文一致性



扩展案例：多Agent RAG

MCP网络搜索工具定义与注册

```
1 # MCP-Search Web and Save Local
2 mcp_client1 = lazyllm.tools.MCPClient(command_or_url="python", args=["-m", "mcp_server_fetch"],)
3 search_agent = CustomReactAgent(llm=lazyllm.OnlineChatModule(source="sensenova", stream=False),
4     stream=False, custom_prompt=search_prompt, tools=mcp_client1.get_tools())
5
6 @fc_register("tool")
7 def search_web(query: str):
8     '''
9     Perform targeted web content retrieval using a combination of search terms and URL.
10    This tool processes both natural language requests and specific webpage addresses
11    to locate relevant online information.
12    Args:
13        query (str): Combined input containing search keywords and/or target URL
14                    (e.g., "AI news from https://example.com/tech-updates")
15    '''
16    query += search_prompt
17    res = search_agent(query)
18    return res
```



扩展案例：多Agent RAG

RAG工具定义与注册+应用编排

```
1 # RAG-Retriever
2 documents = Document(dataset_path='path/to/kb', manager=False)
3 documents.add_reader('*.json', process_json)
4 with pipeline() as ppl_rag:
5     ppl_rag.retriever = Retriever(documents, Document.CoarseChunk,
6     similarity="bm25", topk=1, output_format='content', join='='*20)
7 @fc_register("tool")
8 def search_knowledge_base(query: str):
9     '''
10     Get info from knowledge base in a given query.
11     Args:
12     query (str): The query for search knowledge base.
13     '''
14     res = ppl_rag(query)
15     return res
16 # Agentic-RAG:
17 tools = ['search_knowledge_base', 'search_web']
18 with pipeline() as ppl:
19     ppl.retriever = CustomReactAgent(lazyllm.OfflineChatModule(stream=False), tools, agent_prompt,
20     stream=False)
21     ppl.formatter = (lambda nodes, query: dict(context_str=nodes, query=query)) | bind(query=ppl.input)
22     ppl.llm = lazyllm.OfflineChatModule(stream=False).prompt(lazyllm.ChatPrompter(gen_prompt,
23     extra_keys=["context_str"]))
24 # Launch: Web-UI
25 lazyllm.WebModule(ppl, port=range(23467, 24000), stream=True).start().wait()
```



扩展案例：多Agent RAG

模型结构

```
<Module type=Action return_trace=False sub-  
category=Flow type=Pipeline items=[]>  
  L- <Flow type=Pipeline items=['retriever', 'show',  
    'formatter', 'llm']>  
    |- <Module type=CustomReactAgent>  
    |- <Function type=show>  
    |- <function <lambda> at 0x7fe6e5c4a320>(bind  
args:)<  
    L- <Module type=OnlineChat  
url=https://api.sensenova.cn/v1/llm/ stream=False  
return_trace=False>
```

☐ 使用上下文

☒ 流式输出

☒ 追加输出

处理日志

Clear history

添加新会话

选择会话:

(1) Write five bullets on the latest data r

删除当前会话

Chatbot

Write five bullets on the latest data releases and policy developments in the US economy.

Sources:

<https://www.bea.gov/>

- **U.S. International Trade in Goods and Services, March 2025:** The U.S. monthly international trade deficit increased in March 2025, rising from \$123.2 billion in February (revised) to \$140.5 billion in March, primarily due to a larger increase in imports compared to exports [1].
- **Personal Income and Outlays, March 2025:** Personal income in the U.S. saw an increase of \$116.8 billion (0.5 percent at a monthly rate) in March. Disposable personal income (DPI) rose by \$102.0 billion (0.5 percent), while personal consumption expenditures (PCE) increased by \$134.5 billion (0.7 percent) [2].
- **Gross Domestic Product, 1st Quarter 2025 (Advance Estimate):** Real gross domestic product (GDP) in the U.S. decreased at an annual rate of 0.3 percent in the first quarter of 2025, influenced by an increase in imports and a decrease in exports. This contrasts with a 2.4 percent increase in real GDP in the fourth quarter of 2024 [3].
- **U.S. International Trade in Goods and Services, February 2025:** The U.S. monthly international trade deficit decreased in February 2025, from \$130.7 billion in January (revised) to \$122.7 billion in February, due to increased exports and decreased imports. The goods deficit reduced by \$8.8 billion to \$147.0 billion, while the services surplus decreased by \$0.8 billion to \$24.3 billion [4].
- **Arts and Cultural Production Satellite Account, U.S. and States, 2023:** Arts and cultural economic activity in the U.S. grew by 6.6 percent in 2023, following a 3.8 percent increase in 2022. This sector accounted for 4.2 percent of the GDP, outpacing the broader economy's growth of 2.9 percent in 2023 [5].

References:

1. [U.S. International Trade in Goods and Services, March 2025](#) | 2025-05-06

输入内容并回车!!!



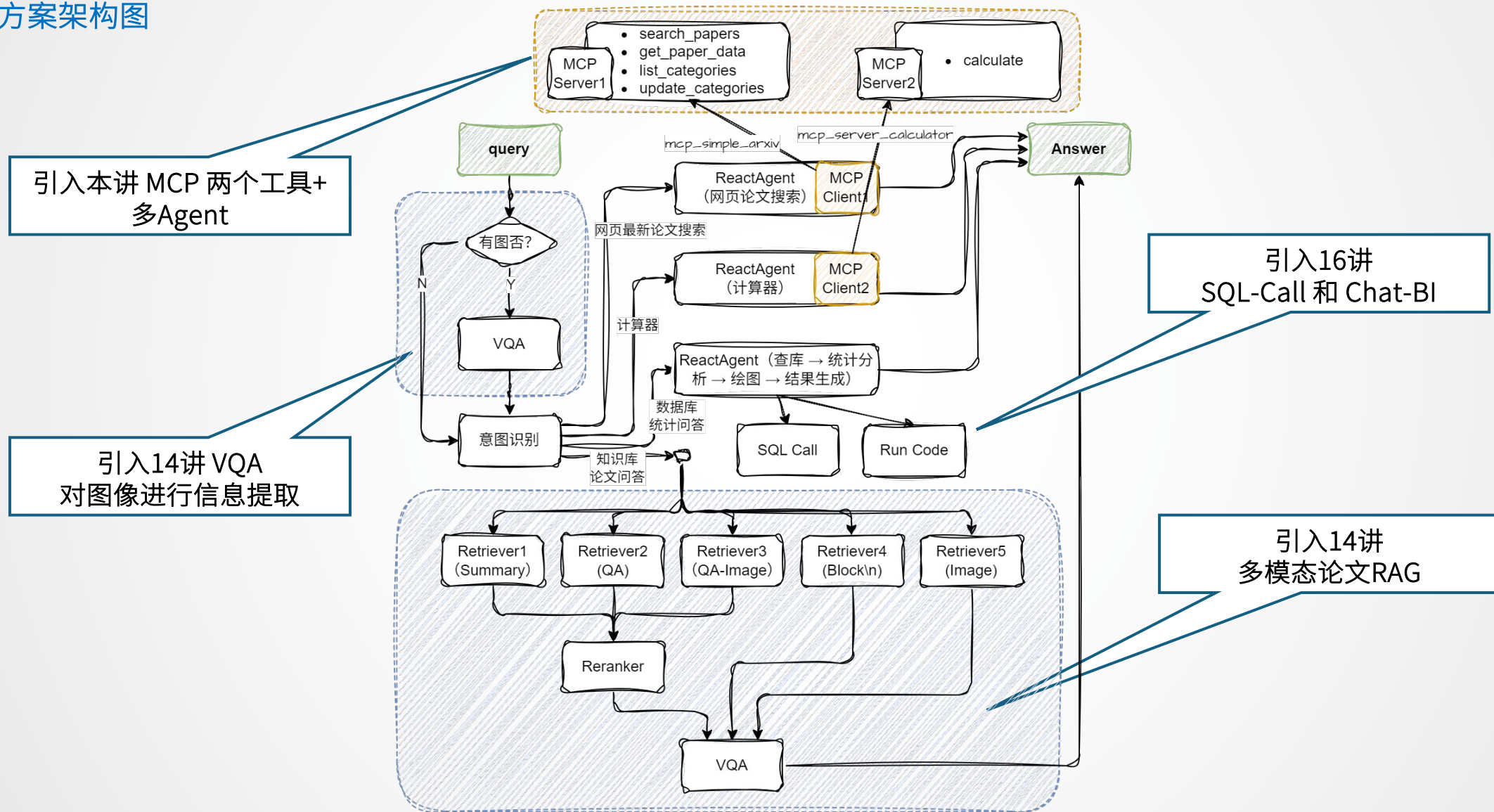
目录

-  1. 上节回顾
-  2. Agentic RAG基本概念
-  3. AI Agent基本概念
-  4. Agentic RAG基础结构及实践
-  5. 多模态Agentic RAG论文系统



多模态Agentic RAG论文系统

方案架构图



多模态Agentic RAG论文系统

配置两个MCP工具及Agent

```
1 import json
2 import lazyllm
3 from lazyllm import ReactAgent
```

用于ArXiv论文搜索

```
4
5
6 mcp_client1 = lazyllm.tools.MCPClient(
7     command_or_url="python",
8     args=["-m", "mcp_simple_arxiv"],
9 )
10 mcp_client2 = lazyllm.tools.MCPClient(
11     command_or_url="python",
12     args=["-m", "mcp_server_calculator"],
13 )
```

用于简单数值计算

```
14
15 llm = lazyllm.OfflineChatModule(stream=False)
```

```
16
17 paper_agent = ReactAgent(llm, mcp_client1.get_tools(), return_trace=True)
18 calculator_agent = ReactAgent(llm, mcp_client2.get_tools(), return_trace=True)
```

环境中需提取安装好两个工具：

- pip install mcp-simple-arxiv
- pip install mcp-server-calculator



多模态Agentic RAG论文系统

应用编排

```
1  # 构建 rag 工作流和统计分析工作流
2  rag_ppl = build_paper_rag()
3  sql_ppl = build_statistical_agent()
4
5  # 搭建具备知识问答和统计问答能力的主工作流
6  def build_paper_assistant():
7      llm = OnlineChatModule(source='qwen', stream=False)
8      vqa = lazyllm.OnlineChatModule(source="sensenova", \
9          model="SenseNova-V6-Turbo").prompt(lazyllm.ChatPrompter(gen_prompt))
10
11     with pipeline() as ppl:
12         ppl.ifvqa = lazyllm.ifs(
13             lambda x: x.startswith('<lazyllm-query>'),
14             lambda x: vqa(x), lambda x: x)
15         with IntentClassifier(llm) as ppl.ic:
16             ppl.ic.case["论文问答", rag_ppl]
17             ppl.ic.case["统计问答", sql_ppl]
18             ppl.ic.case["计算器", calculator_agent]
19             ppl.ic.case["网页最新论文搜索", paper_agent]
20
21     return ppl
22
23 if __name__ == "__main__":
24     main_ppl = build_paper_assistant()
25     lazyllm.WebModule(main_ppl, port=23459, static_paths="./images", encode_files=True).start().wait()
```

• 参照第14讲定义终极多模态 rag pipeline并导入
• 参照16将 定义的SQL-Call和Chat-BI 的 sql pipeline并导入

引入多模态模型
提取图像信息

有图走VQA,
无图就跨过VQA

若意图为“论文问答”,
则通过rag pipeline处理

若意图为“统计问答”,
则通过sql pipeline处理

若意图为“计算器”, 则
调用计算器agent处理

若意图为“网页最新论文搜索”,
则调用论文搜索agent处理

多模态Agentic RAG论文系统



File

Edit

Selection

View

Go

Run

Terminal

Help

sunxiaoye [SSH: SCO-lazyplatform] [Administrator]

statistical_agent.py

mcp_agent.py

run.py

myworks > RAGTutorial > MultiModal > Agentic > run.py

```
25 # 构建 rag 工作流和统计分析工作流
26 rag_ppl = build_paper_rag()
27 sql_ppl = build_statistical_agent()
28
29 # 搭建具备知识问答和统计问答能力的主工作流
30 def build_paper_assistant():
31     llm = OnlineChatModule(source='qwen', stream=False)
32     vqa = lazyllm.OnlineChatModule(source="sensenova",\
33     |     model="SenseNova-V6-Turbo").prompt(lazyllm.ChatPrompter(gen_prompt))
34
35     with pipeline() as ppl:
36         ppl.ifvqa = lazyllm.ifs(
37             lambda x: x.startswith('<lazyllm-query>'),
38             lambda x: vqa(x), lambda x:x)
39         with IntentClassifier(llm) as ppl.ic:
40             ppl.ic.case["论文问答", rag_ppl]
41             ppl.ic.case["统计问答", sql_ppl]
42             ppl.ic.case["计算器", calculator_agent]
43             ppl.ic.case["网页最新论文搜索", paper_agent]
44
45     return ppl
46
47 if __name__ == "__main__":
48     main_ppl = build_paper_assistant()
49     lazyllm.WebModule(main_ppl, port=23459, static_paths="./images", encode_files=True).start().wait()
50
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

GITLENS

(lazyllm)

sunxiaoye @ 69e3d7be-6e7f-11ef-a667-661a8191c638 in ~/myworks/RAGTutorial/MultiModal/Agentic [13:44:04]

\$ pty

1 朋友 2 便宜 3 培养 4 配音 5 拼音 6 平原 7 培育

zsh Agentic

zsh BI-OK

zsh sunxiaoye

Ln 43, Col 49

Spaces: 4

UTF-8

LF

Python



感谢聆听
Thanks for Listening

